

CS 201, Spring 2026

Homework Assignment 3

Due: 23:59, April 24, 2026

1 Introduction

In this homework, you will implement a hospital management system using linked lists. In this system, you will store a linked list of doctors and a linked list of registered patients. This system follows a family doctor model where each patient can be assigned to exactly one primary doctor. Additionally, for each patient, you will maintain two distinct linked lists of medical records: one for current diseases and one for previous (cured) diseases. For each doctor, you will maintain a list of patients who are currently assigned to them. The system should support the following operations:

1. Add doctor
2. Remove doctor
3. Add patient
4. Remove patient
5. Assign a family doctor to a patient
6. Unassign a family doctor from a patient
7. Add a disease to a patient's current record
8. Cure a disease for a patient
9. Show the list of all registered patients
10. Show the list of all doctors in the hospital
11. Show the list of all patients assigned to a doctor
12. Show the complete medical history of a particular patient
13. Show the current diseases of a particular patient

Your implementation must use linked lists for storing the doctors and patients. The doctors and patients must be identified by unique IDs across the system. Note that these ID types are independent of each other. For instance, there cannot be more than one doctor with ID 3, but there can be a doctor with ID 3 and a patient with ID 3 simultaneously. The doctors and patients must be stored in their respective lists. For each doctor, the list of assigned patients must be stored in a linked list. For each patient, both the current disease list and the previous disease list must be stored in linked lists.

2 Implementation Details

addDoctor Adds a new doctor to the hospital with an ID, name, and medical specialty. Each doctor has a unique ID. The system must check whether the specified doctor ID already exists, and if it does, it must not allow the operation and display a warning message.

Example log messages:

-Added doctor 6.

-Cannot add doctor. There is already a doctor with ID 6.

removeDoctor Removes a doctor from the hospital. The system must first check whether the specified doctor ID exists. If it does not, the system must not allow the operation and display a warning message. If the doctor exists, the system must then check whether they currently have patients assigned to them. If they do, the system must not allow the operation and display a warning message.

Example log messages:

- Removed doctor 6.
- Cannot remove doctor. There is no doctor with ID 6.
- Cannot remove doctor. Doctor 6 currently has assigned patients.

addPatient Registers a new patient with an ID and name. The system must check whether the specified patient ID already exists, and if it does, it must not allow the operation and display a warning message.

Example log messages:

- Registered patient 101.
- Cannot register patient. There is already a patient with ID 101.

removePatient Unregisters a patient. If the patient does not exist, is currently assigned to a doctor, or currently has active diseases in their current record, the system must not allow the operation and display a warning message.

Example log messages:

- Unregistered patient 101.
- Cannot unregister patient. There is no patient with ID 101.
- Cannot unregister patient. Patient 101 is still assigned to a doctor or has current diseases.

assignDoctor Assigns a family doctor to a patient. The system must first check whether the specified doctor ID exists. If it does not, it must display a warning message. If the doctor exists, the system must then check whether the specified patient ID exists. If it does not, it must display a warning message. Finally, if both exist, the system should check if the patient is already assigned to a doctor before completing the operation.

Example log messages:

- Patient 101 assigned to doctor 6.
- Cannot assign doctor. There is no doctor with ID 6.
- Cannot assign doctor. Patient 101 already has a family doctor.
- Cannot assign doctor. There is no patient with ID 999.

unassignDoctor Unassigns a family doctor from a patient. The system must first check whether the specified doctor ID exists. If it does not, it must display a warning message. If the doctor exists, the system must then check whether the specified patient ID exists. If it does not, it must display a warning message. Finally, the system must verify that the patient is actually assigned to that specific doctor before proceeding.

Example log messages:

- Patient 101 unassigned from doctor 6.

- Cannot unassign doctor. Doctor 8 does not exist.
- Cannot unassign doctor. Patient 999 does not exist.
- Cannot unassign doctor. Patient 101 does not have doctor 6 as family doctor.

addDisease Adds a disease to a patient's current record. You can assume that disease names start with a capital letter. The system must first check whether the specified patient ID exists. If it does not, it must display a warning message. If the patient exists, the system must then check if the disease is already listed in their current or previous records before adding it to the list.

Example log messages:

- Added disease Flu to patient 101.
- Cannot add disease. There is no patient with ID 999.
- Cannot add disease. Patient 101 already has disease Flu.

cureDisease Moves a disease from a patient's current disease list to their previous disease list. If the disease is successfully cured, it should no longer appear in the current list. The system must first check whether the specified patient ID exists. If it does not, it must display a warning message. If the patient exists, the system must then check whether the specified disease exists in the patient's current disease list. If it does not, it must display a warning message.

Example log messages:

- Cured disease Flu for patient 101.
- Cannot cure disease. There is no patient with ID 999.
- Cannot cure disease. Patient 101 does not have disease Flu in their current records.

showAllPatients Prints the list of all registered patients. If there are no patients, give a warning message. The entries must be in ASCENDING ORDER with respect to the IDs.

Example log messages:

- There are no patients to show.
- Patients in the hospital:
 - Patient 101 : John Doe
 - Patient 102 : Jane Smith

showAllDoctors Prints the list of all doctors. If there are no doctors available, give a warning message. The entries must be in ASCENDING ORDER with respect to the IDs.

Example log messages:

- There are no doctors to show.
- Doctors in the hospital:
 - Doctor 6 : Dr. House (Diagnostic Medicine)
 - Doctor 13 : Dr. Grey (General Surgery)

showDoctorPatients Shows the list of all patients currently assigned to a particular doctor. If the doctor does not exist or has no patients assigned, give a warning message. The patients must appear in the order in which they were assigned to the doctor.

Example log messages:

- Cannot show patients. There is no doctor with ID 13.
- Doctor 13 has no assigned patients.
- Patients of doctor 13:
Patient 101 : John Doe

showPatientHistory Shows the complete medical history of a patient. This includes their ID, name, their current family doctor if any, their current diseases (sorted alphabetically), and their previous diseases (sorted alphabetically).

Example log messages:

- Cannot show history. There is no patient with ID 101.
- Medical History for Patient 101: John Doe
Family Doctor: Dr. House.
Current Diseases: Flu.
Previous Diseases: None.

showCurrentDiseases Shows the list of current diseases for a specific patient in alphabetical order. If the patient does not exist or has no current diseases, show a warning message.

Example log messages:

- Cannot show current diseases. There is no patient with ID 101.
- Patient 101 has no current diseases.
- Current diseases for patient 101: Anemia, Flu.

3.Class Interface and Example Test Code

Below is the required public part of the HospitalSystem class that you must write in this assignment. The interface for the class must be written in the file called HospitalSystem.h and its implementation must be written in HospitalSystem.cpp. You can define additional public and private member functions and data members in this class.

```
class HospitalSystem {  
  
public:  
  
    HospitalSystem();  
  
    ~HospitalSystem();  
  
    void addDoctor(const int doctorId, const string doctorName, const string  
specialty);  
  
    void removeDoctor(const int doctorId);  
  
    void addPatient(const int patientId, const string patientName);  
  
    void removePatient(const int patientId);  
  
    void assignDoctor(const int patientId, const int doctorId);  
  
    void unassignDoctor(const int patientId, const int doctorId);  
  
    void addDisease(const int patientId, const string diseaseName);  
  
    void cureDisease(const int patientId, const string diseaseName);  
  
    void showAllPatients() const;  
  
    void showAllDoctors() const;  
  
    void showDoctorPatients(const int doctorId) const;  
  
    void showPatientHistory(const int patientId) const;  
  
    void showCurrentDiseases(const int patientId) const;  
  
};
```

Example test code:

```
#include "HospitalSystem.h"
#include <iostream>
using namespace std;

int main() {
    HospitalSystem hospital;

    hospital.showAllDoctors();
    hospital.showAllPatients();
    cout << endl;

    hospital.addDoctor(2, "Dr. Grey", "General Surgery");
    hospital.addDoctor(1, "Dr. House", "Diagnostic Medicine");
    hospital.addDoctor(3, "Dr. Strange", "Neurosurgery");
    hospital.addDoctor(2, "Dr. Duplicate", "Cardiology");
    cout << endl;

    hospital.addPatient(102, "Jane Smith");
    hospital.addPatient(101, "John Doe");
    hospital.addPatient(103, "Alice Brown");
    hospital.addPatient(104, "Jack Sea");
    hospital.addPatient(102, "Duplicate Patient");
    cout << endl;

    hospital.showAllDoctors();
    cout << endl;
    hospital.showAllPatients();
    cout << endl;

    hospital.assignDoctor(101, 99);
    hospital.assignDoctor(999, 1);
    hospital.assignDoctor(101, 1);
    hospital.assignDoctor(102, 2);
    hospital.assignDoctor(103, 3);
    hospital.assignDoctor(104, 3);
    hospital.assignDoctor(101, 2);

    cout << endl;

    hospital.showDoctorPatients(99);
    hospital.showDoctorPatients(1);
    hospital.showDoctorPatients(3);
    cout << endl;

    hospital.removeDoctor(99);
    hospital.removeDoctor(1);
    cout << endl;

    hospital.addDisease(999, "Flu");
    hospital.addDisease(101, "Headache");
    hospital.addDisease(101, "Flu");
```

```
hospital.addDisease(102, "Anemia");
hospital.addDisease(101, "Flu");
cout << endl;

hospital.showCurrentDiseases(999);
hospital.showCurrentDiseases(101);
cout << endl;

hospital.removePatient(999);
hospital.removePatient(101);
cout << endl;

hospital.cureDisease(999, "Flu");
hospital.cureDisease(103, "Cold");
hospital.cureDisease(101, "Flu");
hospital.cureDisease(101, "Headache");
cout << endl;

hospital.showCurrentDiseases(101);
cout << endl;

hospital.addDisease(101, "Flu");
cout << endl;

hospital.unassignDoctor(999, 1);
hospital.unassignDoctor(101, 2);
hospital.unassignDoctor(101, 1);
hospital.unassignDoctor(101, 99);
cout << endl;

hospital.showDoctorPatients(1);
cout << endl;

hospital.showPatientHistory(999);
hospital.showPatientHistory(101);
cout << endl;

hospital.showPatientHistory(102);
hospital.cureDisease(102, "Anemia");
hospital.unassignDoctor(102, 2);
hospital.unassignDoctor(103, 3);

hospital.removeDoctor(2);
hospital.removeDoctor(3);
hospital.removeDoctor(1);

hospital.removePatient(101);
hospital.removePatient(102);
hospital.removePatient(103);

return 0;
```

```
}
```

Output of the example test code:

```
There are no doctors to show.
There are no patients to show.

Added doctor 2.
Added doctor 1.
Added doctor 3.
Cannot add doctor. There is already a doctor with ID 2.

Registered patient 102.
Registered patient 101.
Registered patient 103.
Registered patient 104.
Cannot register patient. There is already a patient with ID 102.

Doctors in the hospital:
Doctor 1 : Dr. House (Diagnostic Medicine)
Doctor 2 : Dr. Grey (General Surgery)
Doctor 3 : Dr. Strange (Neurosurgery)

Patients in the hospital:
Patient 101 : John Doe
Patient 102 : Jane Smith
Patient 103 : Alice Brown
Patient 104 : Jack Sea

Cannot assign doctor. There is no doctor with ID 99.
Cannot assign doctor. There is no patient with ID 999.
Patient 101 assigned to doctor 1.
Patient 102 assigned to doctor 2.
Patient 103 assigned to doctor 3.
Patient 104 assigned to doctor 3.
Cannot assign doctor. Patient 101 already has a family doctor.

Cannot show patients. There is no doctor with ID 99.
Patients of doctor 1:
Patient 101 : John Doe
Patients of doctor 3:
Patient 103 : Alice Brown
Patient 104 : Jack Sea

Cannot remove doctor. There is no doctor with ID 99.
Cannot remove doctor. Doctor 1 currently has assigned patients.

Cannot add disease. There is no patient with ID 999.
Added disease Headache to patient 101.
Added disease Flu to patient 101.
Added disease Anemia to patient 102.
Cannot add disease. Patient 101 already has disease Flu.

Cannot show current diseases. There is no patient with ID 999.
Current diseases for patient 101: Flu, Headache.
```


Cannot unregister patient. There is no patient with ID 999.
Cannot unregister patient. Patient 101 is still assigned to a doctor or has current diseases.

Cannot cure disease. There is no patient with ID 999.
Cannot cure disease. Patient 103 does not have disease Cold in their current records.
Cured disease Flu for patient 101.
Cured disease Headache for patient 101.

Patient 101 has no current diseases.

Cannot add disease. Patient 101 already has disease Flu.

Cannot unassign doctor. Patient 999 does not exist.
Cannot unassign doctor. Patient 101 does not have doctor 2 as family doctor.
Patient 101 unassigned from doctor 1.
Cannot unassign doctor. Doctor 99 does not exist.

Doctor 1 has no assigned patients.

Cannot show history. There is no patient with ID 999.
Medical History for Patient 101: John Doe
Family Doctor: None.
Current Diseases: None.
Previous Diseases: Flu, Headache.

Medical History for Patient 102: Jane Smith
Family Doctor: Dr. Grey.
Current Diseases: Anemia.
Previous Diseases: None.
Cured disease Anemia for patient 102.
Patient 102 unassigned from doctor 2.
Patient 103 unassigned from doctor 3.
Removed doctor 2.
Cannot remove doctor. Doctor 3 currently has assigned patients.
Removed doctor 1.
Unregistered patient 101.
Unregistered patient 102.
Unregistered patient 103.

NOTES ABOUT IMPLEMENTATION:

1. You ARE NOT ALLOWED to modify the given parts of the header file. You MUST use **linked lists** in your implementation. You will get no points if you use dynamic or fixed-sized arrays or any other data structures such as vectors/arrays/lists from the standard library. However, if necessary, you may define additional data members and member functions. You may also define additional classes.

2. Moreover, you ARE NOT ALLOWED to use any global variables or any global functions.
3. Output message for each operation MUST match the format shown in the output of the example code. Your output will be compared verbatim with the expected output during evaluation.
4. Your code MUST NOT have any memory leaks. You will lose points if you have memory leaks in your program even though the outputs of the operations are correct. To detect memory leaks, you may want to use Valgrind which is available at <http://valgrind.org>.

NOTES ABOUT SUBMISSION:

1. In this assignment, you must have separate interface and implementation files (i.e., separate .h and .cpp files) for your class. Your class name MUST BE HospitalSystem and your file names MUST BE HospitalSystem.h and HospitalSystem.cpp. Note that you may write additional class(es) in your solution.
2. The code (main function) given above is just an example. We will test your implementation using different scenarios, which will contain different function calls. Thus, do not test your implementation only by using this example code. We recommend you to write your own driver files to make extra tests. However, you MUST NOT submit these test codes (we will use our own test code). In other words, do not submit a file that contains a function called main.
3. You should put all of your .h and .cpp files into a folder and zip the folder (in this zip file, there should not be any file containing a main function). The name of this zip file should conform to the following name convention: secX-Firstname-Lastname-StudentID.zip where X is your section number. The submissions that do not obey these rules will not be graded.
4. Make sure that each file that you submit (each and every file in the archive) contains your name, section, and student number at the top as comments.
5. You are free to write your programs in any environment (you may use Linux, Windows, MacOS, etc.). On the other hand, we will test your programs on “dijkstra.ug.bcc.bilkent.edu.tr” and we will expect your programs to compile and run on the dijkstra machine. If we could not get your program properly work on the dijkstra machine, you would lose a considerable amount of points. We will provide you with example test cases by email. Thus, we recommend you to make sure that your program compiles and properly works on dijkstra.ug.bcc.bilkent.edu.tr before submitting your assignment. If your current code does not fully compile on dijkstra before submission, you can try to comment out the faulty parts so that the remaining code can be compiled and tested during evaluation.
6. You should upload your work to Moodle before the deadline. No hardcopy submission is needed. Late submissions will not be accepted (if you can upload to Moodle, then you are fine). There will be no extension to this deadline.
7. We use an automated tool as well as manual inspection to check your submissions against plagiarism. For questions regarding academic integrity and use of external tools (including generative AI tools), please refer to the course home page and the Honor Code for Introductory Programming Courses (CS 101/102/201/202).

This homework will be graded by your TAs Ahmet Arda Ceylan (arda.ceylan@bilkent.edu.tr) and Pınar Gül (pinar.gul@bilkent.edu.tr). Thus, you may ask your homework related questions directly to them. There will also be a forum on Moodle for questions.